

Smart Nursery Guardian - Software System Flow Report

The Smart Nursery Guardian is an integrated system designed to monitor the baby and the surrounding room environment. The system combines audio processing, machine learning, ESP32 sensors, a graphical user interface (GUI), and Telegram notifications.

The main purpose of the system is to detect whether the baby is crying, determine the possible reason for crying using a machine learning model, monitor room conditions through the ESP32, display the collected information through the GUI, and send alerts when necessary.

The system is divided into several software modules:

- `main.py` – Main coordinator of the entire system.
- `audio.py` – Handles microphone input, noise reduction, and Voice Activity Detection (VAD).
- `ai.py` – Handles machine learning prediction of the baby's crying reason.
- `communication.py` – Handles Bluetooth communication between Python and the ESP32.
- `gui.py` – Provides the graphical user interface.
- `telegrambot.py` – Handles Telegram notifications.
- `cry_reason_rf_model.pkl` – Trained Random Forest machine learning model.

The system starts by running `main.py`. The user does not need to run `audio.py`, `ai.py`, `gui.py`, or `communication.py` separately. These modules are imported and controlled by `main.py`.

1. Introduction

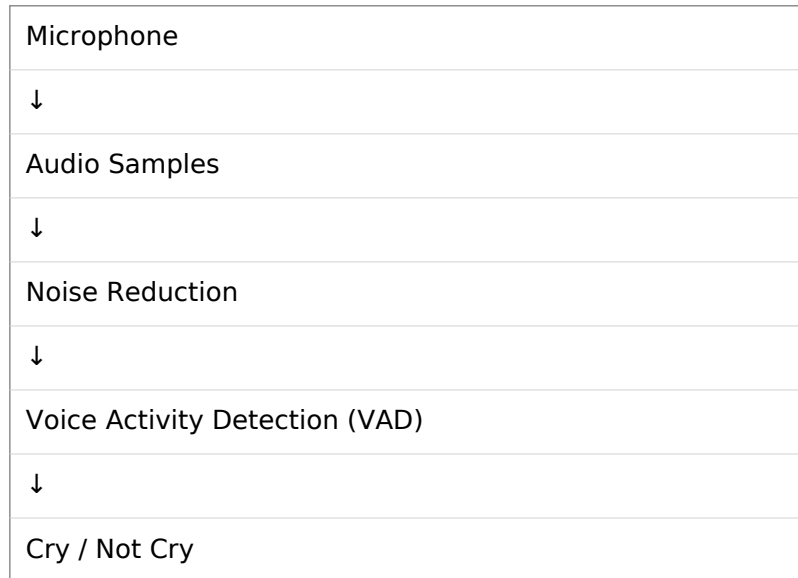
2. System Architecture

3. Main Program Flow

4. Audio Processing Flow

The audio module is responsible for processing the sound captured from the microphone.

The audio pipeline is:



The system continuously receives audio samples from the microphone. The audio processing uses a sample rate of 16,000 Hz and processes audio in small blocks to allow continuous monitoring.

Noise reduction is applied to reduce unwanted background sounds. The system performs an initial calibration period to estimate the background noise. This helps improve the quality of the audio signal before classification.

WebRTC VAD is used to determine whether the current audio contains relevant voice-like activity. The VAD stage helps distinguish active sound from silence or background noise. This result is then passed to main.py.

4.1 Microphone Input

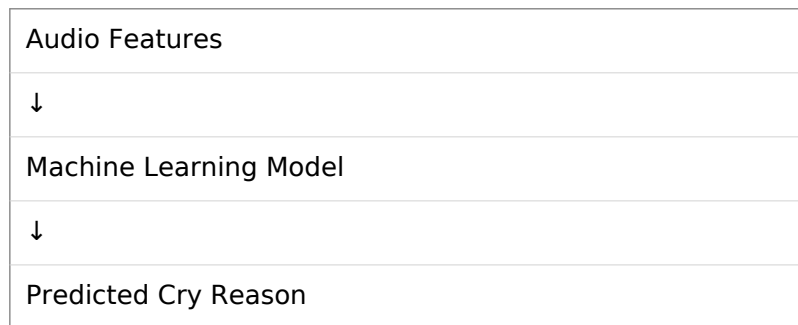
4.2 Noise Reduction

4.3 Voice Activity Detection

5. Machine Learning Flow

The AI module is activated when the audio processing indicates that the baby is crying.

The flow is:



The AI module uses audio features such as MFCC and other relevant audio characteristics to represent the sound numerically. The trained Random Forest model is loaded from `cry_reason_rf_model.pkl`. The model then predicts the possible reason for crying, such as Hungry, Tired, or Discomfort. The predicted result is returned to `main.py`.

5.1 Handling the Imbalanced Data Issue

Since the given data was extremely balanced towards hungry, more than 500 copies were made to help the model recognize the tired and hungry classes. The model was also provided with as many useful features as possible, because the augmented copies will not have the same effect as the many original copies of the hungry class.

Exactly 13 Mel-Frequency Cepstral Coefficients (MFCCs) are extracted to express each audio file numerically. The mean and standard deviation, in addition to the mean of the delta MFCCs, are also used. Other extracted audio features include Root Mean Square (RMS) energy, Zero Crossing Rate (ZCR), and Spectral Centroid. All the previous feature-extraction work is done using the `librosa` library in Python.

These features help the model understand how each class is represented and make up for the needed data.

5.2 Data Augmentation

The data augmentation techniques applied include:

- Time stretching – modifying speed between 0.9 and 1.1.
- Pitch shifting – adjusting between -2 and 2 steps.
- Noise addition.
- Combined augmentation – applying all three techniques together.

Augmentation is applied only to the training data, which represents 80% of the data, and only to the tired and discomfort audio samples.

5.3 Random Forest Classifier

The classification model is built using a Random Forest Classifier (`RandomForestClassifier`) from the `sklearn.ensemble` library.

The model is configured with 300 estimators (`n_estimators=300`), a maximum depth of 8 (`max_depth=8`), and a minimum samples split of 5 (`min_samples_split=5`). It uses the entropy criterion to evaluate splits and applies balanced class weights to further account for any remaining label imbalances.

5.4 Prediction Thresholds

Thresholds are used for the minority classes to help recognize them. If the model's predicted probability for the tired class reaches 0.30 or higher, the script classifies the audio as tired. The same approach is used for discomfort, but its threshold is set to 0.35.

If neither of these specific thresholds is met, the logic defaults to a hungry classification.

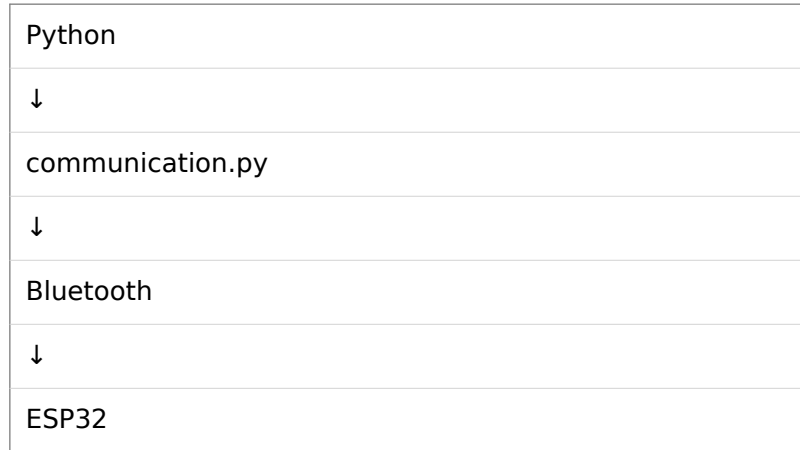
Upon completion, the fully trained Random Forest model is exported and saved to disk as `cry_reason_rf_model.pkl` using the `joblib` library.

6. Communication with ESP32

The communication module is responsible for connecting Python with the ESP32 through Bluetooth.

The connection is handled through communication.py. The ESP32 connection uses a serial Bluetooth port, for example COM4, with the configured baud rate.

The communication flow is:



The ESP32 provides environmental and baby-related sensor information.

- Temperature
- Gas status
- Baby awake/status information
- Other room sensor values depending on the implemented firmware

The communication module allows main.py to request and receive these values.

7. GUI Integration

The graphical interface is implemented in gui.py and initialized by main.py.

The basic software flow is:



The GUI provides a simple and organized control and monitoring interface for the Smart Nursery Guardian. It displays the current system status and is designed so that sensor data, baby conditions, alerts, and messages can be updated easily when connected to the actual monitoring system and ESP32.

7.1 Main Window and Status Information

A main window titled "Smart Nursery Guardian" is created with a simple and organized layout.

The Baby and Room Status section displays:

- Temperature
- Baby status
- Cry status
- Cry type
- Gas status
- Light status
- ESP32 connection status

7.2 Video, Alerts, and Messages

A separate Video section shows the current video status. Separate Alerts and Messages sections are provided for displaying system notifications.

7.3 GUI Controls and Update Functions

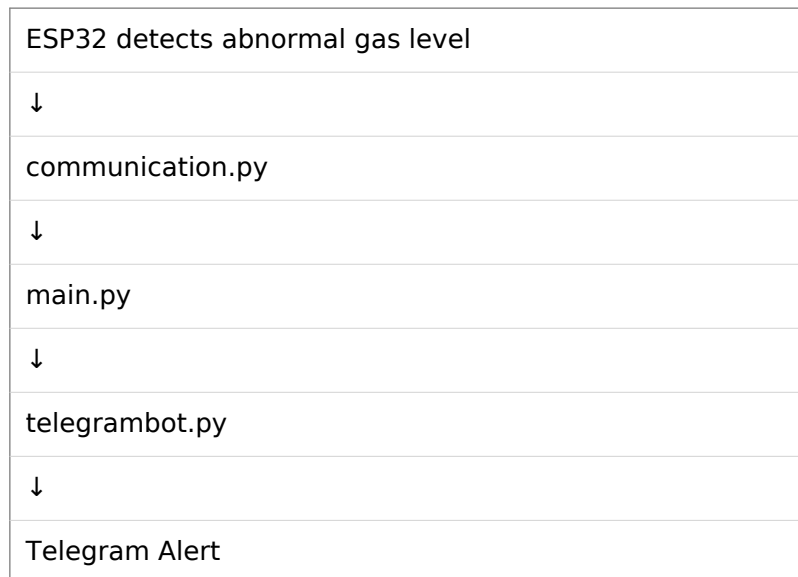
The interface includes buttons to Clear alerts, Clear messages, and Exit the application.

Update functions allow other parts of the system to change the displayed sensor and baby information. Additional functions show alerts and messages dynamically.

8. Telegram Notification System

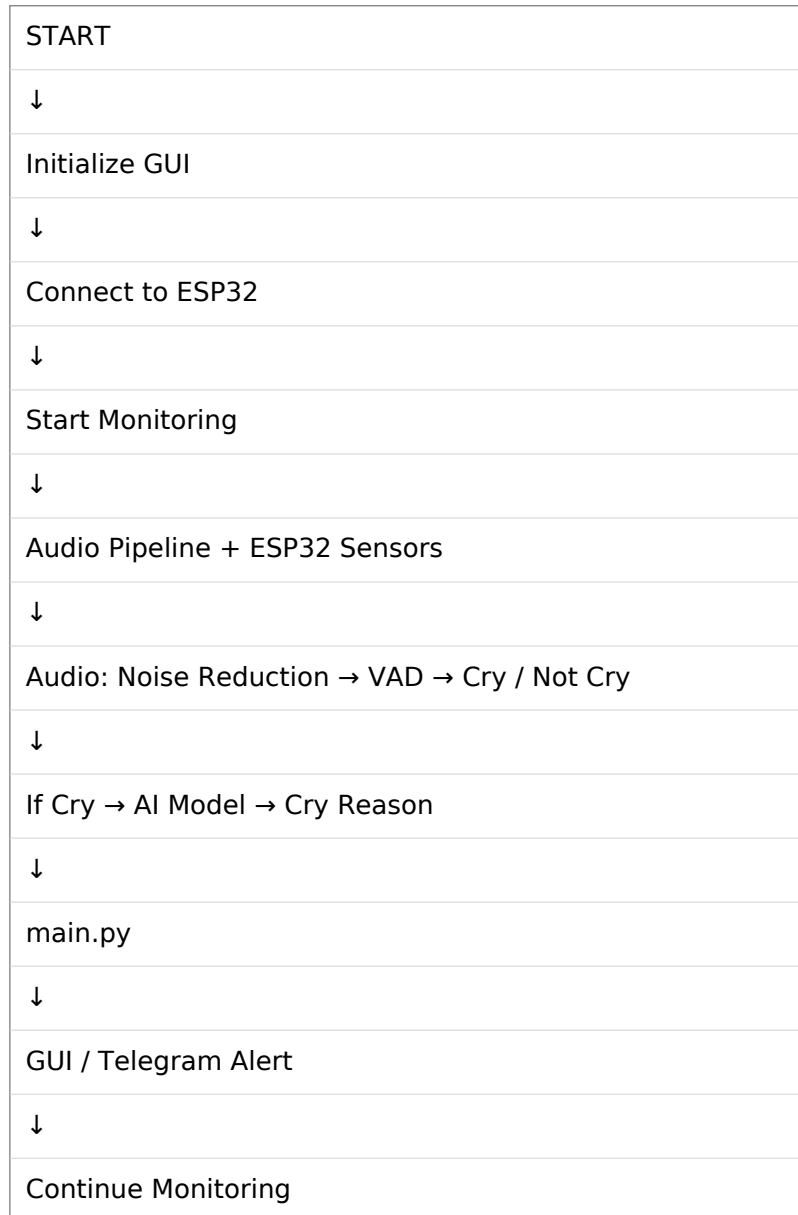
The Telegram functionality is implemented separately in telegrambot.py. The main program calls the Telegram functions when an important event occurs.

For example:



This keeps the Telegram implementation separate from the main program while allowing main.py to control when an alert should be sent.

9. Complete System Flow



10. Role of main.py

main.py acts as the central coordinator of the project. It does not need to contain every implementation detail of every module. Instead, it connects the modules together and controls the overall flow.

Its main responsibilities are:

- Starting the application
- Initializing the GUI
- Connecting to the ESP32
- Requesting audio results
- Calling the AI model when crying is detected
- Receiving sensor information
- Updating the GUI

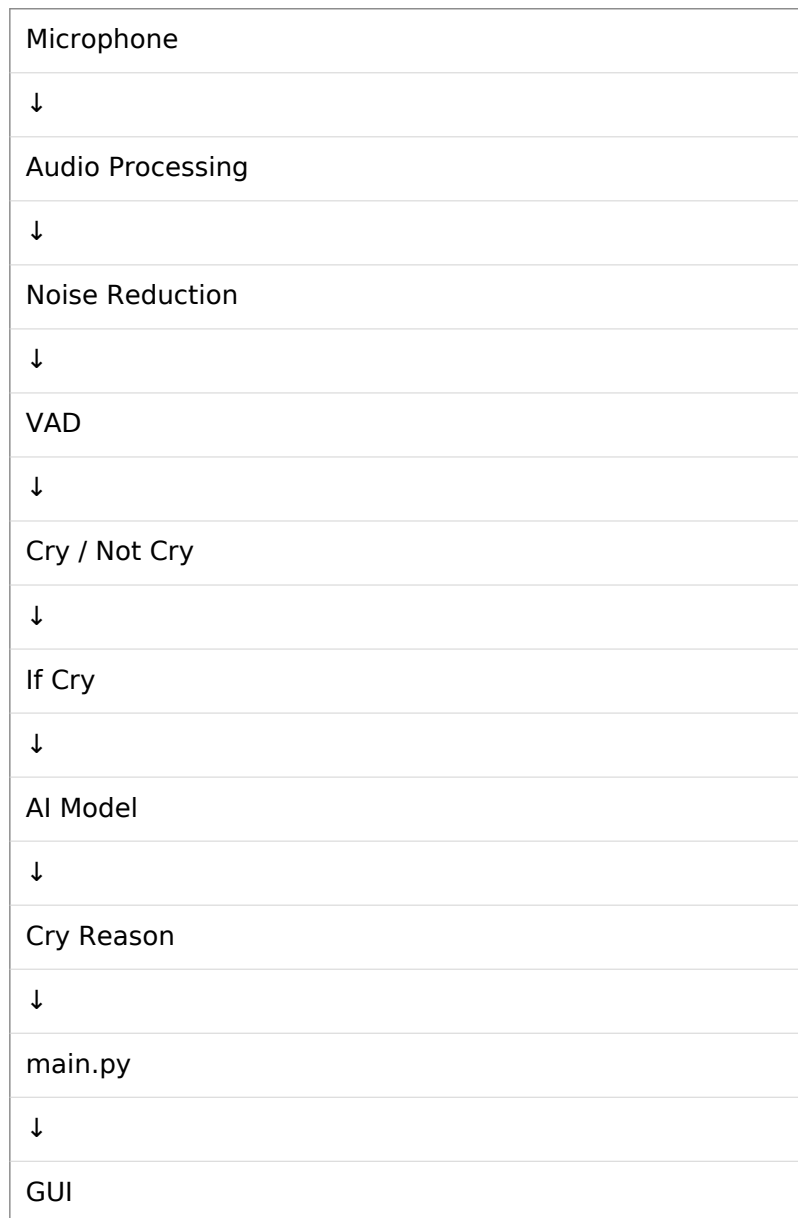
- Triggering Telegram alerts
- Keeping the system running continuously

Therefore, the overall concept is:

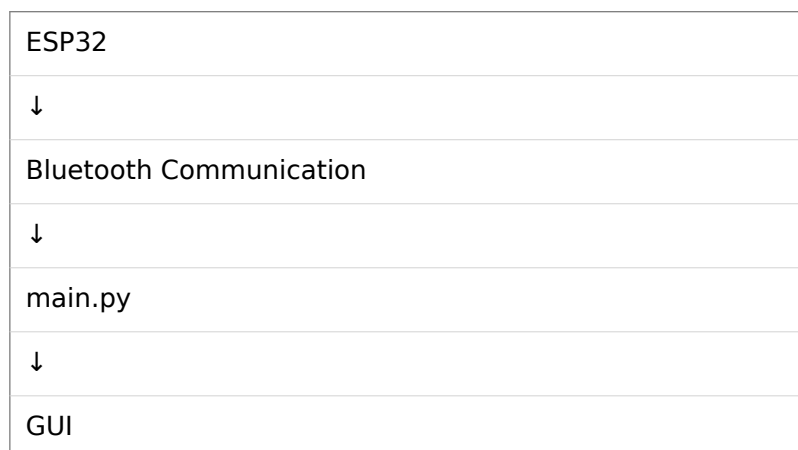
- main.py = Coordinator
- audio.py = Audio Processing
- ai.py = Machine Learning
- communication.py = ESP32 Communication
- gui.py = User Interface
- telegrambot.py = Notifications

11. Final Workflow

The final workflow of the Smart Nursery Guardian is:



At the same time:



If an important condition is detected:



This integration allows the system to continuously monitor the baby and the room, process audio intelligently, communicate with the ESP32, display system information through the GUI, and notify the user when necessary.